ari-conformance-README

ARI Conformance Kit

Executable proof that a runtime satisfies the Agent Runtime Interface.

This is the **third artifact** a standard needs (alongside the spec and a reference implementation): a test corpus a *third party* can run against their own runtime to make a credible, falsifiable conformance claim. See `docs/ari-strategy.md` for why this matters.

Each test maps to a numbered requirement in ARI-SPEC.md and to a box in its Appendix B checklist.

Layout

File	Profile	Needs the built extension?
`test_ari_core.py`	ARI-Core (§2–§7)	yes
`test_ari_persistence.py`	persistence round-trip (§8, §2.2)	yes
`test_ari_embedded_runtime.py`	ARI-Embedded runtime checks (§10.2)	yes
`test_ari_embedded_structural.py`	ARI-Embedded structural checks (§10.2)	**no**

Tests that need the compiled `agentcore` extension `importorskip` it, so the **structural** checks (no managed-runtime dependency in the core, near-zero dependency footprint, embeddable C++ example present) run anywhere — even on a machine with no C++ toolchain.

Run it

```
# Full kit (requires the extension built — see repo README "Install")
pip install -e ".[test]"
pytest ari-conformance/ -v

# Structural-only (no build needed)
pytest ari-conformance/test_ari_embedded_structural.py -v
```

In this repo's CI, the kit runs against the freshly-built extension on Linux — see the `conformance` job in `.github/workflows/ci.yml`.

Claiming conformance

Per ARI §10.4, a conformance claim MUST name the profile(s), the spec version, and the kit version, e.g.:

> **ARI 0.1 · Core + Embedded · kit v0.1 · 24/24 passing**

Do **not** advertise "ARI-compatible" without a named profile and version.

Status

Kit v0.1. The reference implementation (`agentcore`) is the first runtime under test. Until a green run is confirmed in CI, the CI job is marked `continue-on-error` (informational) — flip it to a hard gate once validated.